Exercise: Pairwise Distances Under a Learned Metric

Written by Chris and Claude

November 1, 2026

Read this first

Please don't spend any more than 8 hours on this. The goal is not to "finish", it's to try something new that relates to the Einsummable project and then to meet and talk about it, not to get the best results. When the clock runs out, stop and write up what you have.

You may use AI tools as much as you like. In fact, it is required! However: *you must understand everything in your write-up and everything in the code it describes.* Use AI to help you make progress, but make sure you understand everything that is going on. The point here is to undertake a project in an area that you probably don't know much about, and to learn about it and make some progress.

The deliverable that matters is a **conversation**. We'll meet and spend 30-45 minutes talking through your write-up and results with Chris and one of the graduate students on the project. We may pick pieces of your code and ask you to walk us through them. We will ask why one optimization helped, and why the next one didn't help as much as you expected.

We are not evaluating whether you get the very best possible output! We are evaluating whether you made progress and understand what you did, and what you learned.

1 Why this exercise

While we don't often write kernels in Einsummable, we are always worried about performance: how data is tiled, where it lives in the memory hierarchy, how often it is reused, and what the hardware's ceiling is in terms of performance. This exercise asks you to develop exactly that kind of thinking on a small, well-defined CPU problem. The ideas (register blocking, cache blocking, packing, fusing cheap epilogues into expensive loops) all translate into the Einsummable project.

2 The problem

You are given a data matrix $X \in \mathbb{R}^{n \times d}$ (one data point per row) and a linear map $M \in \mathbb{R}^{d \times k}$ learned by some metric-learning procedure we don't care about. Define the transformed points $y_i = x_i M \in \mathbb{R}^{1 \times k}$. The task is to compute the $n \times n$ matrix D of *squared* distances between all pairs of transformed points:

$$D_{ij} \;=\; \|x_i M - x_j M\|_2^2 \;=\; (x_i - x_j)\, M M^\top (x_i - x_j)^\top.$$

This is the squared Mahalanobis distance under the metric $A = MM^\top$.

Interface. Implement, in C or C++,

```
void pairwise_sqdist(int n, int d, int k,
                     const double *X, /* n x d, row-major, leading dim d */
                     const double *M, /* d x k, row-major, leading dim k */
                     double *D);      /* n x n, row-major, leading dim n */
```

You may fill all of D or only its upper triangle (including the diagonal). Say which in your write-up. You may allocate scratch memory inside the function. Everything is double precision.

Correctness. D must agree with an exact reference to within a relative tolerance of 10^{-9} on each entry, where the reference scale for entry (i, j) is $\max(1, \|y_i\|^2 + \|y_j\|^2)$. The starter harness (Section 3) checks this two ways and prints `PASS` or `FAIL` per kernel: a direct evaluation of $\|(x_i - x_j)M\|^2$ in extended precision on two thousand random pairs (the `sampled` column), and a comparison of every entry you claim to fill against a BLAS-based implementation (the `full` column). Your kernel must pass both for every size in the benchmark grid before its timing counts. The harness also reports how many sampled entries came back *negative* (the `neg` column). A negative squared distance is not a failure, but it is telling you something; see Section 6.

Performance. Single-threaded. Do not use OpenMP or any other threading; the whole exercise is about one core. The harness times each kernel after one warm-up run and reports the minimum and median of five runs, plus GFLOP/s computed as *useful work* divided by the minimum time. "Useful work" is fixed by the harness (`useful_flops()` in `src/util.c`) as the operation count of the cheaper of the two obvious matrix-product formulations, counting each unordered pair once. Every kernel is scored against that same number, so a kernel that does more work than necessary is penalized accordingly. If you quote a GFLOP/s figure in your write-up that is not the harness's, say exactly what you divided by.

3 The environment

Do all of your benchmarking on the Rice **CLEAR** cluster (`ssh clear.rice.edu`). Develop wherever you like.

- The CLEAR nodes are dual-socket Intel Xeon E5-2640 v4 (Broadwell-EP): 10 physical cores per socket, 2 hyperthreads per core (so `lscpu` reports 40 CPUs), 2.4 GHz base, 3.4 GHz max turbo, 32 KiB L1d and 256 KiB L2 per core, 25 MiB L3 per socket, two NUMA nodes. The ISA is **AVX2 with FMA**—256-bit vectors, 16 YMM registers, two FMA units per core—*not* AVX-512. Run `lscpu` yourself and confirm this before you write a single intrinsic. Everything in the tutorial transfers; the vector width and register count are what change, and working out what that does to the micro-kernel shape is part of the exercise.
- The harness builds with `gcc -O3 -march=native`, which enables AVX2 and FMA on these nodes. The naive baseline is additionally compiled with `-fno-tree-vectorize` so that it really is scalar; your kernels are not, so if you want to know what the compiler is doing for you, look at the assembly (`gcc -S` or `objdump -d`).
- The reference BLAS is OpenBLAS, which is already installed on CLEAR (`/usr/include/cblas.h` plus `libopenblas.so.0`; the build script finds and links it automatically, falling back to FlexiBLAS if needed). The harness uses it for the full-matrix check, as the kernel named `blas` so you can see what a vendor library achieves on this exact task, and for `./bench`

`--ceiling`, which times a square `dgemm` as the practical single-core ceiling. You are not required to beat `blas`. You are required to know how far from it you are and why.

- CLEAR is a shared, interactive machine. Your timings will be noisy. The grid script pins your process to one core (`taskset`, plus `numactl` where available); if you run `./bench` by hand, pin it yourself. Even-numbered CPUs are socket 0 and odd-numbered are socket 1, and CPU i and CPU i+20 are hyperthreads of the same physical core, so a busy sibling will halve your throughput. Check `uptime` before a run, benchmark when the machine is quiet, and watch the gap between the minimum and median times: when they disagree by more than a few percent, someone else was using the core. The harness records the load average in every CSV row.
- The starter harness is at [repo URL]. Use it; eight hours is not enough time to write plumbing. Appendix A is a one-page reference.

Benchmark grid. Fix $d = 512$. Run every combination of

$$n \in \{1024,\ 4096,\ 8192\}, \qquad k \in \{32,\ 128,\ 512\}.$$

Nine configurations. The point of varying k is that the right algorithm—and the right bottleneck—changes with it. Part of what we want to see is whether you noticed.

4 What to do

Step 0: algebra before code. The formula as written suggests a loop: for each pair (i, j), form $x_i - x_j$, multiply by M, take the norm. Work out the operation count of that approach. Then work out at least two algebraically equivalent reformulations that reorganize the computation around matrix products, and derive the cost of each as a function of n, d, k. Decide which to implement for which part of the grid, and say why. This should take you well under an hour (especially with AI help) and is the single most important step.

Step 1: read the tutorial, selectively. Work through `https://github.com/yzhaiustc/Optimizing-DGEMM-on-Intel-CPUs-with-AVX512F` (linked under "How to optimize DGEMM on x86 CPU platforms"). It builds a double-precision matrix multiply in eighteen steps, from a naive triple loop to something that matches Intel MKL. Kernels 1–9 are the ones that apply directly to our hardware: register blocking, AVX2 SIMD, loop unrolling, an 8×4 micro-kernel, cache blocking, and packing. Kernels 10–11 redo the same ideas with AVX-512, but you cannot run them on CLEAR. Kernels 12–18 (inline assembly, software prefetching, frequency effects) are interesting, but you do not have time for them.

Step 2: build a ladder. Starting from the naive implementation in the harness, produce a sequence of kernels, each one adding a single idea, and measure each on the full grid. Each rung is its own file in `src/kernels/` plus one line in `registry.c`. Keep every version! A plausible ladder, in rough order of payoff:

1. Naive direct evaluation (provided as `k0`). Establish the baseline and confirm your cost model. The harness refuses to run it where its estimated cost exceeds 4×10^{11} flops, and warns you before any run that will take more than a few seconds. Before you reach for `--force`, work out from your cost model which grid points it will skip, and check your prediction against what it does.

2. Your chosen algebraic reformulation, written with plain loops. This alone should be orders of magnitude faster.
3. A register-blocked, SIMD micro-kernel for the dominant matrix product.
4. Cache blocking and packing around that micro-kernel.
5. *Then* the things that are specific to this problem and not in the tutorial. We will not list them all, but here are two to get you thinking: the output matrix is symmetric, and the expensive step almost certainly ends with a cheap per-element correction that is being applied in a separate pass over n^2 doubles. Ask yourself what each of those facts is worth, in flops or in bytes, before you spend time on it.

You will probably get through three or four rungs. That is fine.

Step 3: compare to the ceiling. Compute the theoretical single-core peak of your CLEAR node in double-precision GFLOP/s from the clock rate, the number of FMA units, and the vector width. Measure OpenBLAS `dgemm` at a large square size (`./bench --ceiling`) as the *practical* ceiling; on an idle CLEAR core this comes out between 41 and 43 GFLOP/s, so if you measure something very different, the machine was busy. Reconcile that figure with the theoretical peak you computed; the difference is not all OpenBLAS's fault. Place every rung of your ladder, on every grid point, relative to those two numbers and to the `blas` kernel. Then for your best kernel, make an argument—not a guess—about what is limiting it: compute, memory bandwidth, latency, something else. The `blas` kernel's own distance from the `dgemm` ceiling, and how that distance changes with k, is a good place to start thinking.

5 Deliverables

Email a link to a repository containing the following to [grad student email]; we will set up the meeting from there.

1. **Code.** The harness directory with every rung of your ladder registered in `src/kernels/registry.c`, such that `scons` builds it on CLEAR with no edits and `./bench --list` shows your kernels in ladder order.
2. **Results.** The `results.csv` produced by `scripts/run_grid.sh` over the full grid, and the figure `scripts/plot.py` makes from it (useful GFLOP/s per kernel, one panel per k, one line per n, the `dgemm` ceiling as a dashed line). You may restyle the figure or make your own; the data must be the CSV's.
3. **A write-up of at most two pages** (plus the figure), with these sections, in this order:
   (a) *Algorithm and cost model.* The reformulation(s) you chose, their operation counts, and the crossover reasoning.
   (b) *The ladder.* One short paragraph per rung: what changed, what you expected, what you measured, and—if they differ—your best explanation.
   (c) *Where you are and why.* Peak and practical ceilings, where your best kernel sits, and your argument for what bounds it.
   (d) *What you would do with eight more hours*, in order of expected payoff, with a sentence of justification each.
   (e) *How you used AI.* What you asked for, what you kept, what you threw out and why. Be concrete. We are not grading this section; we are reading it to understand how you work.
   (f) *A rough time log.* Half-hour granularity is fine. Where did the eight hours go?

6 A few more things

- **Useful flops.** Report the harness's GFLOP/s numbers; they all share one denominator, so kernels are comparable. If you quote any other operation count (the naive one, a BLAS routine's, your own), say exactly what you divided by.
- **Numerics.** At least one natural reformulation can return slightly negative "squared distances" for nearly coincident points, and the harness's test data contains a few dozen such pairs on purpose (look at `gen_data` in `src/util.c`). If the `neg` column is nonzero for your kernel, handle it and explain the mechanism in your write-up. If it is zero, think about why.
- **Full disclosure!** If something doesn't work, say so and say what you think went wrong. A clear account of a failed rung is one of the most useful things you can show us.

We look forward to the conversation.

A Harness quick reference

Build. Requires SCons (`pip install --user scons` if `which scons` fails; it lands in `~/.local/bin`) and gcc.

```
scons                # prints the BLAS it found ("*** BLAS: <cblas.h> + ...")
scons -c             # clean
./bench --list       # kernels, in registry order, with "upper"/"full" and a description
./bench --help       # every option
```

```
./bench --kernel k0 --n 1024 --d 512 --k 32        # one kernel, one size
./bench --kernel k0,k2,blas --n 4096 --d 512 --k 128 # several kernels
./bench --ceiling                                   # OpenBLAS dgemm 2048^3, one core
scripts/run_grid.sh results.csv k0 k2 k3 blas       # whole grid + ceiling -> CSV, pinned
scripts/plot.py results.csv ladder.png              # the figure
```

Useful options: `--reps R` (default 5), `--no-full-check` while iterating (the BLAS reference at $n = 8192$ takes a few seconds and 512 MB), `--no-check` for timing-only runs, `--force` to run a kernel the harness thinks is too expensive. The grid script accepts `CORE=`, `N_LIST=`, `K_LIST=`, `REPS=` and `EXTRA=` as environment variables.

Output columns. `check` (PASS/FAIL), `sampled` and `full` (worst relative error seen by each check), `neg` (sampled entries that were negative), `t_min` and `t_med` (seconds), `GFLOP/s` (useful work / `t_min`), `load` (one-minute load average at the time of the run).

Adding a rung.

1. Copy `src/kernels/k1_yours.c` to `src/kernels/k2_<idea>.c`; rename the function.
2. Add one line to `src/kernels/registry.c`:
   `{ "k2", kernel_k2, fills_full, NULL, "what it does" },`
   Set `fills_full` to 0 if you only write D_{ij} for $j \geq i$; the checker then ignores the lower triangle.
3. `scons`. The build globs `src/kernels/`; nothing else changes.

The interface is documented at the top of `include/harness.h`. You may allocate scratch space inside your kernel; its cost is part of your time.

What the data looks like. Rows of X come from eight Gaussian clusters whose scales span two orders of magnitude, so distances range from $\sim 10^{-13}$ (the planted near-duplicates) to $\sim 10^{4}$. M has i.i.d. $N(0, 1/d)$ entries. Everything is seeded; every run sees the same data.